

Bhai, ye question LeetCode ka ekdum practical aur mast question hai ("**Time Needed to Buy Tickets**").

Pehle iski simple simulation yaani **Brute Force Approach** ko ekdum real life ke example se samajhte hain.

1. Real-Life Example Se Question Samjho

Maan lo ek ticket counter par line lagi hai: `tickets = [2, 3, 2]` aur tumhe index $k = 1$ wale bande (jiski value 3 hai) ka total time nikalna hai.

Rules:

1. Ek baar me ek hi banda counter par jayega aur **sirf 1 ticket** kharidega.
2. Ticket kharidne me **1 second** lagta hai.
3. Agar uski tickets bachi hain, toh wo line ke **sabse aakhiri me** wapas khada ho jayega.
4. Agar uski tickets khatam (0) ho gayi, toh wo line se bahar nikal jayega.

Simulation (Brute Force Kaise Chalega):

- **Round 1:**
 - Index 0 (2 tickets): 1 ticket li (ab bachi 1), line ke piche gaya. **Time = 1s**
 - Index 1 (3 tickets - *Hamara K*): 1 ticket li (ab bachi 2), line ke piche gaya. **Time = 2s**
 - Index 2 (2 tickets): 1 ticket li (ab bachi 1), line ke piche gaya. **Time = 3s**
 - *Line ki nayi halat:* [1, 2, 1]
 - **Round 2:**
 - Index 0 (1 ticket): 1 ticket li (ab bachi 0), line se bahar. **Time = 4s**
 - Index 1 (2 tickets - *Hamara K*): 1 ticket li (ab bachi 1), line ke piche gaya. **Time = 5s**
 - Index 2 (1 ticket): 1 ticket li (ab bachi 0), line se bahar. **Time = 6s**
 - *Line ki nayi halat:* [0, 1, 0]
 - **Round 3:**
 - Index 1 (1 ticket - *Hamara K*): 1 ticket li (ab bachi 0). **Time = 7s**
 - **STOP!** Kyunki hamare k -th bande ki saari tickets khatam ho gayi, hum wahin ruk jayenge. Total time aaya **7 seconds**.
-

2. Brute Force Approach (The Array Simulation)

Brute Force ka simple matlab hai: "**Jaise queue chal rahi hai, exact code me waisa hi loop ghumate jao tab tak jab tak k -th element 0 na ho jaye.**"

Hum ek `while` loop chalayenge jo tab tak chalega jab tak `tickets[k] > 0` hai. Uske andar ek `for` loop chalega jo puri line ke har bande ko ek-ek karke check karega.

3. Brute Force C++ Code

```
#include <iostream>
#include <vector>
using namespace std;

class Solution {
public:
    int timeRequiredToBuyBrute(vector<int>& tickets, int k) {
        int n = tickets.size();
        int totalTime = 0;

        // Jab tak hamare k-th bande ki tickets khatam nahi hoti, line ghumate
raho
        while (tickets[k] > 0) {

            // Puri line par ek baar sweep maaro
            for (int i = 0; i < n; i++) {

                // Agar is bande ko ticket chahiye (tickets > 0)
                if (tickets[i] > 0) {
                    tickets[i]--; // 1 ticket kharidi
                    totalTime++; // 1 second laga
                }

                // Beech loop me check karo, kya hamare k-th bande ka kaam ho
gaya?
                // Agar haan, toh aage ke logon ko ticket dilane ki need nahi,
yahi se ruk jao!
                if (tickets[k] == 0) {
                    return totalTime;
                }
            }
        }

        return totalTime;
    }
};

int main() {
    vector<int> tickets = {2, 3, 2};
    int k = 1; // 1st index wale bande ka time nikalna hai (value = 3)

    Solution sol;
    cout << "Brute Force Total Time: " << sol.timeRequiredToBuyBrute(tickets, k)
    << " seconds" << endl;
    // Output: 7 seconds
}
```

```
    return 0;  
}
```

4. Complexity Analysis (Yeh Kyun Bura Hai?)

1. Time Complexity: $O(N \times M)$

- Jahan N array ka size hai aur M tickets ki maximum value hai (jaise `tickets[i]` ki value ho sakti hai 100).
- Kyun?** Kyunki agar kisi bande ko 100 tickets chahiye, toh hamara `while` loop lagbhag 100 baar chalega, aur har baar andar wala loop N baar ghumega.
- Agar array ka size 100 ho aur ticket count 100 ho, toh $100 \times 100 = 10,000$ operations. Waise toh ye LeetCode par pass ho jata hai kyunki constraints chote hain, par interviewer ko ye approach **bohot ghatiya** lagti hai kyunki hum falthu me baar-baar subtract kar rahe hain.

2. Space Complexity: $O(1)$

- Kyun?** Kyunki humne koi extra memory ya Queue data structure use nahi kiya. Humne original array me hi values ko ghatiya (modify) kiya hai.

5. Un Chaaro Edge Cases Par Yeh Brute Force Kaise Chalega?

- Negative Elements:** Ticket count kabhi negative nahi ho sakta (hamesha ≥ 1 hoga), toh ye case problem nahi karega.
- Ek Hi Element** (`tickets = [5], k = 0`): `while` loop chalega, `for` loop sirf 1 element par chalega. 5 baar ghum kar `totalTime = 5` de dega aur ruk jayega. **(PASS ✓)**
- Zero Present Ho** (`[2, 0, 3], k = 2`): Hamare code me `if (tickets[i] > 0)` check laga hai. Toh jab ye 0 par aayega, use skip kar dega, na time badhega na ticket kam hogi. Ek dum **(PASS ✓)**
- Duplicate Elements** (`[2, 2, 2], k = 1`): Saare log barabar tickets le rahe hain, loop standard sequencing follow karega aur perfectly **(PASS ✓)** hoga.

Bhai, brute force ekdum mast samajh aya? Isko bina array ko baar-baar ghumaaye, sirf ek single loop me $O(N)$ **Optimal approach** se kaise karte hain (bina simulation ke dimaag lagakar maths se), wo bataun?

Bhai, ab aate hain is question ke **Better Approach** par.

Pichle Brute Force wale tareeqe me sabse badi kharabi kya thi? Hum array me baar-baar (`while` loop chala kar) ghum rahe the aur elements ko ek-ek karke ghaat (subtract) rahe the. Agar kisi ko 100 ticket chahiye, toh hum 100 baar us index par ja rahe the, jo ki bilkul falthu ka repetition hai.

Better Approach ka simple logic ye hai: Hum bina array ko baar-baar ghumaaye, **Queue Data Structure** ka use karke is pure process ko exact real life ki tarah simulate karenge. Isme hum har bande ke index ko queue me daal denge aur jab tak hamara kaam nahi hota, unhe aage-piche karte rahenge.

1. Better Logic (Queue Simulation)

Hum ek standard `queue<int>` banayenge jo array ke saare indices (0 se $n-1$) ko apne andar hold karegi.

Steps Kaise Kaam Karenges?

1. Shuruat me queue me saare indices daal do: 0, 1, 2, ..., $n-1$.
2. Ek `totalTime = 0` variable rakho.
3. Jab tak queue khaali nahi hoti, tab tak ek loop chalao:
 - Queue ke sabse aage khade bande ka index nikal lo: `int frontPerson = q.front(); q.pop();`
 - Us bande ne 1 ticket kharidi, toh uska ticket count 1 kam kar do: `tickets[frontPerson]--`
 - Time ko 1 second badha do: `totalTime++`
 - **Check 1:** Agar ye banda hamara target banda tha (`frontPerson == k`) aur iski tickets ab 0 ho chuki hain, toh boss! Hamara kaam ho gaya, yahi se `totalTime` return kar do.
 - **Check 2:** Agar ye target banda nahi tha, ya target banda tha par iski tickets abhi bhi bachi hain (`tickets[frontPerson] > 0`), toh isko line ke sabse aakhiri me dobara khada kar do: `q.push(frontPerson).`

2. Better C++ Code ($O(N \times M)$ with Queue)

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

class Solution {
public:
    int timeRequiredToBuyBetter(vector<int>& tickets, int k) {
        queue<int> q;
        int n = tickets.size();
        int totalTime = 0;

        // Step 1: Saare bandon ke indices ko queue me daal do
        for (int i = 0; i < n; i++) {
            q.push(i);
        }

        // Step 2: Queue ko simulate karo
        while (!q.empty()) {
            int frontPerson = q.front(); // Sabse aage wale bande ko uthaya
            q.pop();

            tickets[frontPerson]--; // Usne 1 ticket li
            totalTime++;           // 1 second lag gaya

            // Agar hamara k-th banda apni saari tickets le chuka hai, toh ruk jao
            if (frontPerson == k && tickets[k] == 0) {
                return totalTime;
            }
        }
    }
};
```

```

        // Agar is bande ki tickets abhi bhi bachi hain, toh line ke piche
        bhejo
        if (tickets[frontPerson] > 0) {
            q.push(frontPerson);
        }
    }

    return totalTime;
}
};

int main() {
    vector<int> tickets = {2, 3, 2};
    int k = 1; // 1st index wale bande ka time nikalna hai

    Solution sol;
    cout << "Better Approach Total Time: " << sol.timeRequiredToBuyBetter(tickets,
k) << " seconds" << endl;
    // Output: 7 seconds

    return 0;
}

```

3. Complexity Analysis

- **Time Complexity:** $O(N \times M)$
- **Kyun?** Kyunki abhi bhi hum mathematically operations ko kam nahi kar paa rahe hain. Agar tickets ki value M hai, toh har element queue me baar-baar ghusta aur nikalta rahega. Toh total time complexity worst case me $N \times \text{max_ticket_value}$ hi rahegi.
- **Space Complexity:** $O(N)$ ❌
- **Kyun?** Kyunki humne yahan ek extra `queue<int>` banayi hai jisme humne array ke saare indices store kiye hain.

Dhyan dene wali baat: Yeh approach brute force se zyada "clean" aur "logical" hai kyunki ye memory me exact real queue bana rahi hai. Lekin, isne extra space ($O(N)$) le li jo brute force me $O(1)$ thi.

4. Un Chaaro Edge Cases Par Yeh Code Kaise Chalega?

1. **Negative Elements:** Ticket count hamesha ≥ 1 hoga, toh negative ka koi chance hi nahi hai.
2. **Ek Hi Element ([5], k=0):** Queue me sirf 0 index jayega. Wo niklega, ticket 4 hogi, wapas push hoga. Aise 5 baar chal kar perfectly **PASS** ✅ ho jayega.
3. **Zero Present Ho ([2, 0, 3], k=2):** Shuru me hi agar hum queue me push karte waqt check laga dein ya fir queue se nikalte waqt, toh zero wale log turant pop ho jayenge bina time kharab kiye. Humne code me `if (tickets[frontPerson] > 0)` laga kar unhe piche bhejna band kar diya hai, isliye **PASS** ✅ ho jayega.

4. **Duplicate Elements ([2, 2, 2], k=1)**: Queue standard order maintain rakhegi, duplicates aapas me bina kisi clash ke ek-ek karke process honge. **PASS** ✓

Bhai, ye Queue wali simulation approach achhi hai, par iska asli jadoo **Optimal Approach** me hai, jahan bina koi array ya queue ghumaye, sirf **Maths ka dimaag laga kar single loop me $O(N)$ Time aur $O(1)$ Space** me answer aa jata hai. Wo dikhaun ab?

Bhai, ab aate hain is question ke **Absolute Optimal Approach** par. Yeh approach bina kisi array ya queue ko ghumaye, sirf **ek single loop ($O(N)$ Time) aur $O(1)$ Space** me maths ka dimaag laga kar seedha answer nikal deti hai.

Interviewer tumse pakka yahi approach expect karega kyunki ye bina kisi simulation ke chalti hai.

1. Optimal Logic: Bina Queue Ke Answer Kaise Nikalein?

Socho, hamara target banda kaun hai? Index k . Usko total kitni tickets chahiye? $tickets[k]$. Jab tak index k wale bande ka kaam khatam nahi hota, tab tak line ke baaki log kya kar rahe honge?

Hum puri line ke logon ko do hisson me baant sakte hain:

Hissaa 1: Jo log k se pehle khade hain ($i \leq k$)

- Yeh log line me k se aage hain, toh inko k ke saath-saath barabar mauka milega.
- Agar kisi bande ko k se zyada ticket chahiye, toh wo k ke poore hone tak maximum $tickets[k]$ baar hi counter par ja payega.
- Agar kisi ko k se kam ticket chahiye (jaise sirf 1 ticket), toh wo apni tickets lekar line se hat jayega, yaani $tickets[i]$ baar jayega.
- **Formula kya bana?** $\min(tickets[i], tickets[k])$

Hissa 2: Jo log k ke baad khade hain ($i > k$)

- Yeh log line me k ke piche hain. Jab k apna aakhiri ticket le raha hoga, tab ye piche wale log counter tak nahi pahunch payenge! Isliye inka aakhiri round miss ho jayega.
- Toh ye log k ke poore hone tak maximum $tickets[k] - 1$ baar hi counter par ja payenge.
- Agar kisi ko usse bhi kam ticket chahiye, toh wo apni saari tickets le lega, yaani $tickets[i]$ baar jayega.
- **Formula kya bana?** $\min(tickets[i], tickets[k] - 1)$

2. Optimal C++ Code ($O(N)$ Time, $O(1)$ Space)

Hume bas ek single loop chalana hai aur har element ke liye upar wala logic joddte jana hai:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

class Solution {
```

```

public:
    int timeRequiredToBuyOptimal(vector<int>& tickets, int k) {
        int totalTime = 0;
        int n = tickets.size();
        int targetTickets = tickets[k];

        // Sirf EK SINGLE LOOP
        for (int i = 0; i < n; i++) {

            // Case 1: Jo log k se pehle ya k khud hain
            if (i <= k) {
                totalTime += min(tickets[i], targetTickets);
            }
            // Case 2: Jo log k ke baad khade hain
            else {
                totalTime += min(tickets[i], targetTickets - 1);
            }
        }

        return totalTime;
    }
};

int main() {
    vector<int> tickets = {2, 3, 2};
    int k = 1; // 1st index wale bande ka time nikalna hai (value = 3)

    Solution sol;
    cout << "Optimal Approach Total Time: " <<
    sol.timeRequiredToBuyOptimal(tickets, k) << " seconds" << endl;
    // Output: 7 seconds

    return 0;
}

```

3. Complexity Analysis

- **Time Complexity:** $O(N)$ 🔥
- **Kyun?** Kyunki humne array ko sirf **ek baar** scan kiya hai. Na koi **while** loop hai, na koi simulation. Array ka size chahe 10^5 ho aur tickets 10^9 ho, ye code ek hi baar me answer de dega.
- **Space Complexity:** $O(1)$ 🔥
- **Kyun?** Kyunki humne koi extra data structure (jaise Queue ya alag array) use nahi kiya. Humne sirf ek variable **totalTime** banaya hai jo constant space leta hai.

4. Edge Cases Par Iska Test

Chalo dekhte hain hamara ye mathematical hero edge cases ko kaise dho dheta hai:

1. Ek Hi Element (`tickets = [5], k = 0`):

- Loop sirf `i = 0` par chalega. `i <= k` (`0 <= 0`) true hai. `totalTime += min(5, 5) → totalTime = 5`. Perfect! **(PASS ✓)**

2. Zero Present Ho (`tickets = [2, 0, 3], k = 2`):

- Jab loop `i = 1` (value 0) par aayega, toh `min(0, 3)` ya `min(0, 2)` hamesha 0 hi joddega. Yaani zero ticket wale log automatically safe tarike se ignore ho jayenge. **(PASS ✓)**

3. Duplicate Elements (`tickets = [2, 2, 2], k = 1`):

- `targetTickets = 2`.
- `i=0 (<=k): min(2, 2) = 2`
- `i=1 (<=k): min(2, 2) = 2`
- `i=2 (>k): min(2, 2-1) = min(2, 1) = 1`
- Total = $2 + 2 + 1 = 5$ seconds. Simulation ke mutabik bhi yahi answer aayega. **(PASS ✓)**

Bhai, ab samajh aaya ki bina queue banaye sirf math laga kar kaise is question ko $O(N)$ me nichoda ja sakta hai?

Bhai, "Time Needed to Buy Tickets" ke is mathematical code ko lekar interviewer tumhe phansane ki poori koshish karega. Wo tumse poochega ki *"Bhai, bina queue ke chalne wala ye code alag-alag tedhe-medhe inputs par sahi chalega ya crash ho jayega?"*

Chalo, is optimal code ke **saare edge cases aur hidden traps** ka poora dry-run karke dekhte hain taaki tum interviewer ke samne ekdum confident raho.

Hamara optimal logic ek baar yaad kar lo:

- Agar banda k se pehle ya khud k hai (`i <= k`): `totalTime += min(tickets[i], targetTickets)`
- Agar banda k ke baad hai (`i > k`): `totalTime += min(tickets[i], targetTickets - 1)`

1. Target Banda Line me Sabse Pehle Ho ($k = 0$)

Situation: Maan lo line hai `tickets = [3, 4, 2]` aur hamara target banda $k = 0$ (value = 3) hai. Yani pehla banda hi hamara target hai.

Dry Run:

- `targetTickets = tickets[0] = 3`
- `i = 0 (i <= k): totalTime += min(tickets[0], 3) → min(3, 3) = 3. totalTime = 3`.
- `i = 1 (i > k): totalTime += min(tickets[1], 3 - 1) → min(4, 2) = 2. totalTime = 3 + 2 = 5`.
- `i = 2 (i > k): totalTime += min(tickets[2], 3 - 1) → min(2, 2) = 2. totalTime = 5 + 2 = 7`.
- **Result:** Output **7 seconds** aaya.
- **Real Check:** Agar line physically chalti, toh $k=0$ teesri baar ticket lekar hat jata. Uske piche wale log sirf 2 hi rounds dekh paate. Toh $3 + 2 + 2 = 7s$. Ekdum **PASS ✓**

2. Target Banda Line me Sabse Aakhiri Ho ($k = n - 1$)

Situation: Maan lo line hai `tickets = [2, 1, 3]` aur hamara target banda $k = 2$ (value = 3) hai, jo sabse piche khada hai.

Dry Run:

- `targetTickets = tickets[2] = 3`
- Is case me saare elements ke liye $i \leq k$ ($0 \leq 2, 1 \leq 2, 2 \leq 2$) sach hoga! Yaani `else` condition kabhi chalegi hi nahi.
- $i = 0$: `min(2, 3) = 2`
- $i = 1$: `min(1, 3) = 1`
- $i = 2$: `min(3, 3) = 3`
- `totalTime = 2 + 1 + 3 = 6` seconds.
- **Result:** Kyunki target banda sabse piche hai, uske aage khade sabhi logon ko poora mauka milega apni tickets khatam karne ka jab tak k ka kaam chal raha hai. Isliye ye pure calculation ko bina single round miss kiye jodta hai. Ek dum **PASS** ✓

3. Baaki Log Target Bande Se Bohot Zyada Tickets Chahte Hon (`tickets[i] >> tickets[k]`)

Situation: Maan lo `tickets = [100, 2, 100]` aur hamara $k = 1$ (value = 2) hai. Yaani beech wale ko sirf 2 ticket chahiye, par aage piche wale bhoot hain, unhe 100-100 tickets chahiye!

Dry Run:

- `targetTickets = 2`
- $i = 0$ ($i \leq k$): `min(100, 2) = 2`
- $i = 1$ ($i \leq k$): `min(2, 2) = 2`
- $i = 2$ ($i > k$): `min(100, 2 - 1) = min(100, 1) = 1`
- **Result:** Total Time = $2 + 2 + 1 = 5$ seconds.
- **Real Check:** k ke 2 ticket lene tak, index 0 wala 2 ticket le chuka hoga, k khud 2 le chuka hoga, aur index 2 wala sirf 1 ticket le payega (kyunki wo k ke piche hai, aur k ke 2nd ticket lete hi line ruk jayegi). Total: $2 + 2 + 1 = 5$. Mathematical formula ne bina loop lagaye bade numbers ko ek jhatke me clip kar diya! Ek dum **PASS** ✓

4. Array me Zeros Present Hon (`tickets = [4, 0, 2]`, $k = 0$)

Situation: Maan lo kisi bande ko 0 tickets chahiye (yaani wo line me sirf maze lene khada hai, use ticket counter par rukna hi nahi hai).

Dry Run:

- `targetTickets = 4`
- $i = 0$ ($i \leq k$): `min(4, 4) = 4`
- $i = 1$ ($i > k$): `min(0, 4 - 1) = min(0, 3) = 0` → Isne total time me 0 add kiya!
- $i = 2$ ($i > k$): `min(2, 4 - 1) = min(2, 3) = 2`

- **Result:** Total Time = $4 + 0 + 2 = 6$ seconds.
- **Reality:** 0 wale bande par counter ka koi time kharab nahi hoga, hamara math use automatically 0 jod kar skip kar deta hai. Ek dum **PASS** ✓

5. Target Bande Ko Sirf 1 Ticket Chahiye Ho ($\text{tickets}[k] = 1$)

Interviewer ka Mind Game: "Bhai, agar target bande ko sirf 1 hi ticket chahiye, toh $\text{tickets}[k] - 1$ toh 0 ho jayega! Kya wahan tumhara code phansega?"

Situation: $\text{tickets} = [5, 1, 4]$, $k = 1$ (value = 1)

- $\text{targetTickets} = 1$
- $i = 0$ ($i \leq k$): $\min(5, 1) = 1$
- $i = 1$ ($i \leq k$): $\min(1, 1) = 1$
- $i = 2$ ($i > k$): $\min(4, 1 - 1) = \min(4, 0) = 0$
- **Result:** Total Time = $1 + 1 + 0 = 2$ seconds.
- **Reality:** Bilkul sahi hai! Agar pehle bande ne 1 ticket li (1s), fir k ne apni 1 ticket li (1s)—jaise hi k ki ticket khatam hui, game khatam! k ke piche wale bande ($i = 2$) ko ek baar bhi counter par jaane ka mauka nahi mila, isliye uska contribution 0 aaya. Logic ek dum **PASS** ✓

Cheat-Sheet: Interviewer Ke Saamne Bolne Ke Liye

Scenario	Edge Case Ka Twist	Hamara Code Kaise Handle Karta Hai?
$k = 0$ (Sabse Aage)	Piche walo ko 1 round kam milna chahiye.	$i > k$ wali condition unhe automatically $\text{targetTickets} - 1$ par limit kar deti hai.
$k = n-1$ (Sabse Piche)	Sabhi ko poore rounds milne chahiye.	$i \leq k$ hamesha true rehta hai, sabko full targetTickets tak ka mauka milta hai.
Bade Numbers (10^9)	Infinite Loops ka darr (Simulation fail).	Hum loop chala hi nahi rahe, direct $\min()$ function se value utha rahe hain.
Zeros ($\text{tickets}[i] = 0$)	Counter par time waste nahi hona chahiye.	$\min(0, \text{target})$ hamesha 0 return karta hai, koi fake time add nahi hota.
$\text{tickets}[k] = 1$	Piche walo ko 0 mauka milna chahiye.	$\text{targetTickets} - 1 = 0$ ho jata hai, jisse piche walo ka contribution automatically zero ho jata hai.

Bhai, ab is "Time Needed to Buy Tickets" ke optimal approach aur iske saare edge cases par tumhari aisi pakad ho gayi hai ki interview me tum samne wale ko dry-run karke shock kar sakte ho!

Kaho, ek dum makkhan ki tarah clear hai na ab?

Implement Stack USING Queue

Bhai, ye question interviewer ka ek aur bohot favourite dimaag ghumane wala question hai—"Implement Stack using Queues".

Stack ka funda hota hai **LIFO** (Last In First Out - jo aakhiri me aayega, wo pehle jayega) aur Queue ka funda hota hai **FIFO** (First In First Out - jo pehle aayega, wo pehle jayega). Ab hame ek seedhe-saadhe FIFO queue se zabardasti LIFO stack ka kaam karwana hai!

Chalo iska ekdum basic **Brute Force Approach** samajhte hain jisme hum **Do Queues (Two Queues)** ka use karenge.

1. Brute Force Ka Logic (Do Queues Ka Khel)

Is approach me hum do queues banate hain:

1. **q1**: Yeh hamari **Main Queue** hogi jo hamesha stack ke elements ko sahi order me hold karegi.
2. **q2**: Yeh hamari **Helper Queue** hogi jo naye element ko sahi jagah pahunchane me madad karegi.

Hum stack ke **push()** operation ko **bhari (expensive)** banayenge, taaki **pop()** aur **top()** ekdum makkhan ($O(1)$) chalein. Jab bhi koi naya element push hoga, hum poori queue ko palti marwayenge!

Push Kaise Karenge? (Step-by-step)

Maan lo stack me **q1** me pehle se elements hain. Ab naya element **X** aaya:

1. Sabse pehle naye element **X** ko khaali helper queue **q2** me daal do (**q2.push(X)**). Ab **X** **q2** ke sabse aage baith gaya.
2. Ab main queue **q1** me jitne bhi purane elements hain, unhe ek-ek karke nikaalo (**q1.pop()**) aur **q2** me **X** ke piche daalte jao (**q2.push()**).
3. Ab hamari **q2** me saare elements ekdum perfect stack wale order me aa chuke hain (naya element sabse aage aa gaya).
4. Aakhiri me, dono queues ka naam aapas me badal (swap) do, taaki **q1** fir se hamari main queue ban jaye aur **q2** agli baar ke liye khaali ho jaye!

2. C++ Code (Brute Force - Two Queues)

```
#include <iostream>
#include <queue>
using namespace std;

class MyStackBrute {
private:
    queue<int> q1; // Main Queue
    queue<int> q2; // Helper Queue

public:
    MyStackBrute() {}

    // 1. PUSH Operation: O(N) Time (Expensive)
    void push(int x) {
```

```
// Step 1: Naye element ko helper queue q2 me daalo
q2.push(x);

// Step 2: q1 ke saare elements ko q2 me transfer karo
while (!q1.empty()) {
    q2.push(q1.front());
    q1.pop();
}

// Step 3: q1 aur q2 ko swap kar do
// Ab q1 fir se main queue ban gayi aur q2 khaali ho gayi
swap(q1, q2);
}

// 2. POP Operation: O(1) Time
int pop() {
    if (q1.empty()) return -1;
    int topElement = q1.front();
    q1.pop(); // Sabse aage se nikal diya (jo aakhiri me aaya tha)
    return topElement;
}

// 3. TOP Operation: O(1) Time
int top() {
    if (q1.empty()) return -1;
    return q1.front();
}

// 4. EMPTY Operation: O(1) Time
bool empty() {
    return q1.empty();
}
};

int main() {
    MyStackBrute st;
    st.push(10);
    st.push(20);
    st.push(30);

    cout << "Stack Top: " << st.top() << endl; // Output: 30 (LIFO)
    cout << "Popped: " << st.pop() << endl;    // Output: 30
    cout << "New Stack Top: " << st.top() << endl; // Output: 20

    return 0;
}
```

3. Complexity Analysis

- **Time Complexity:**

- **push(x): $O(N)$ ❌** (Kyuki har baar naya element aane par hum N elements ko ek queue se nikaal kar doosri me daal rahe hain).
- **pop() / top() / empty(): $O(1)$ 🔥** (Ek jhatke me main queue ke front se element mil jata hai).
- **Space Complexity: $O(N)$**
- **Kyun?** Kyuki hum total N elements ko store karne ke liye do queues ka use kar rahe hain (bhalerao dono milkar N elements hi hold karti hain).

4. Edge Cases Par Yeh Kaise Chalega?

1. Khaali Stack Se Pop/Top Karna (Underflow):

- Agar stack khaali hai aur koi **pop()** ya **top()** karega, toh hamare code me check laga hai **if (q1.empty()) return -1;**, jo crash hone se bacha lega aur **-1** return kar dega. **(PASS ✅)**

2. Ek Hi Element Hona:

- **push(10)** karne par **q2** me **10** jayega, **q1** khaali hai toh loop nahi chalega, swap hoke **q1** me **10** aa jayega. **pop()** karne par wahi wapaa mil jayega. Perfectly **(PASS ✅)**

3. Duplicate Elements (push(10), push(10)):

- Queue duplicates ko unke order ke mutabik handle karti hai. Dono **10** sahi sequence me push aur pop honge. **(PASS ✅)**

Bhai, do queues wala ye brute force approach dimaag me ghusa? Isko aur zyada optimize karne ke liye interviewer bolta hai: "*Bhai, do queues toh koi bhi use kar leta hai, kya tum ise **sirf EK single queue (One Queue)** se $O(N)$ push me implement kar sakte ho?*" — Wo wali better approach dekhni hai ab?

Bhai, ab aate hain is question ke **Absolute Optimal Approach** par.

Interview me interviewer hamesha brute force ke baad tumse yahi boleگا: "*Bhai, do queues use karke toh sabhi solve kar lete hain, kya tum bina koi extra space liye, **sirf EK single queue (One Queue)** se Stack implement kar sakte ho?*"

Haan, bilkul kar sakte hain! Single queue se jab hum Stack implement karte hain, toh hamari Space Complexity $O(N)$ **se ghat kar ekdum minimal** ho jati hai (kyuki hum dusri helper queue ka karch bachate hain) aur push operation abhi bhi $O(N)$ rehta hai, jabki baaki saare functions $O(1)$ me chalte hain.

1. Optimal Logic: Ek Queue Se Palti Kaise Maarein? (The Rotation Trick)

Socho, queue me elements linear order me ghuste hain. Agar hume naye element ko sabse aage (front par) lana hai taaki wo Stack ke LIFO order ko follow kare, toh hum **Queue Rotation** ka use karte hain.

Khel Kaise Chalta Hai? (Step-by-Step)

Maan lo queue me pehle se 2 elements hain: **[20, 10]** (Yahan **20** front par hai, yaani ye stack ka top hai). Now, naya element aaya **30**.

- Step 1:** Naye element **30** ko chupchap queue me push kar do (`q.push(30)`).
- Ab queue dikhegi aise: `[20, 10, 30]` (Yahan **20** front par hai aur **30** back par chala gaya).
- Step 2:** Lekin stack ke niyam ke mutabik **30** ko toh sabse aage hona chahiye tha! Toh hum kya karenge? Naye element se pehle jitne bhi purane elements the (yaani queue ka purana size N), unhe ek-ek karke front se nikalenge (`pop`) aur wapas piche push (`push`) karte jayenge!
 - Purana size tha 2.
 - Rotation 1:** Front se **20** ko nikala aur piche daal diya. Queue bani: `[10, 30, 20]`
 - Rotation 2:** Front se **10** ko nikala aur piche daal diya. Queue bani: `[30, 20, 10]`
- Magic!** Dhyan se dekho, naya element **30** ab automatically queue के **sabse aage (front)** aa chuka hai, aur baaki elements ka order bhi sahi hai.

2. Optimal C++ Code (Using Only 1 Queue)

```
#include <iostream>
#include <queue>
using namespace std;

class MyStackOptimal {
private:
    queue<int> q; // Sirf EK single queue

public:
    MyStackOptimal() {}

    // 1. PUSH Operation: O(N) Time, O(1) Extra Space (Expensive par Smart)
    void push(int x) {
        // Naye element ko aane se pehle queue ka current size nikal lo
        int currentSize = q.size();

        // Step 1: Naye element ko normal push karo (ye piche jayega)
        q.push(x);

        // Step 2: Purane saare elements ko front se utha kar back me rotation
        // maaro
        for (int i = 0; i < currentSize; i++) {
            q.push(q.front()); // Front wale ko piche bheja
            q.pop();           // Front se delete kiya
        }
    }

    // 2. POP Operation: O(1) Time
    int pop() {
        if (q.empty()) return -1;
        int topElement = q.front();
        q.pop(); // Front par hamesha latest element hoga, use direct pop kiya
        return topElement;
    }
}
```

```
// 3. TOP Operation: O(1) Time
int top() {
    if (q.empty()) return -1;
    return q.front(); // Front hi hamara stack top hai
}

// 4. EMPTY Operation: O(1) Time
bool empty() {
    return q.empty();
}

};

int main() {
    MyStackOptimal st;
    st.push(10);
    st.push(20);
    st.push(30);

    cout << "Stack Top: " << st.top() << endl; // Output: 30 (LIFO Order!)
    cout << "Popped: " << st.pop() << endl;    // Output: 30
    cout << "New Stack Top: " << st.top() << endl; // Output: 20

    return 0;
}
```

3. Complexity Analysis

- **Time Complexity:**
- **push(x):** $O(N)$ (Kyuki hum queue ke size ke barabar rotation loop chala rahe hain).
- **pop() / top() / empty():** $O(1)$ 🔥 (Koyi loop nahi, direct front se element mil raha hai).
- **Space Complexity:** $O(1)$ **Extra Space** 🔥
- **Kyun?** Kyuki humne stack ke elements ko store karne ke liye sirf ek hi mandatory queue li hai. Humne brute force ki tarah koi extra helper queue (q2) ya extra temporary memory arrays nahi banaye hain.

4. Un Chaaro Edge Cases Par Iska Performance

Yeh single-queue optimal code ekdum solid hai, chalo ise test karte hain:

1. Underflow Case (Khaali Stack me Pop/Top):

- Code me check laga hai `if (q.empty()) return -1;`. Agar queue khaali hogi toh ye safe tarike se bina crash kiye `-1` de dega. **(PASS ✅)**

2. Ek Hi Element (push(10)):

- `currentSize = 0` hoga. `q.push(10)` chalega. Loop 0 baar ghumega (yani nahi chalega). Queue me `[10]` rahegi. Ek dum perfect. **(PASS ✓)**

3. Duplicate Elements (`push(10)`, `push(10)`):

- Rotation dono elements par unke relative ordering ke mutabik sahi chalega, duplicate hone se conditions par koi farq nahi padta. **(PASS ✓)**

4. Bade Inputs / Continuous Operations:

- Kyunki hum $O(1)$ extra space use kar rahe hain, memory limits ya pointers leak hone ka koi darr nahi hai. Continuous streams me bhi ye micro-seconds me perform karega. **(PASS ✓)**

Bhai, ek single queue se elements ko gol-gol ghuma kar kaise Stack banaya jata hai, ye dhasu optimal trick dimaag me fit baith gayi?

Bhai, tumne ek dum up-level soch lagayi hai! Jab interviewer tumse single queue wala code likhwa leta hai ($O(N)$ push, $O(1)$ pop), toh wo aakhiri me ek googly fenkta hai:

"Kya tum koi aisa tarika nikaal sakte ho jahan PUSH aur POP dono me se koi bhi $O(N)$ na ho? Ya fir PUSH ko $O(1)$ kar sakte ho?"

Iska jawab hai: **Haan, bilkul kar sakte hain!** Lekin iske liye hume thoda sa "Out of the Box" sochna padega. Isko bolte hain **Amortized $O(1)$ ya Standard $O(1)$ Push Approach** jahan hum Queue ke kaam karne ke dhang ko thoda badal dete hain.

Chalo iske dono bade dhasu tarike samajhte hain.

Approach 1: Push ko $O(1)$ banao, Pop ko $O(N)$ (Two Queues)

Agar tumhari application me **Push bohot zyada baar** ho raha hai aur **Pop bohot kam**, toh tum Push ko ek dum fast ($O(1)$) kar sakte ho aur saara load Pop par daal sakte ho.

Logic:

- **Push(x):** Kuch mat soch, naye element ke aate hi use chupchap `q1` me push kar do. Kuch rotate karne ki zaroorat nahi hai. → **Time: $O(1)$**
- **Pop():** Stack ka नियम hai ki sabse aakhiri me aaya banda pehle niklega. Par Queue (`q1`) me wo sabse piche baitha hai. Toh hum `q1` ke saare elements ko nikaal kar `q2` me daalenge, bas **aakhiri wale element ko chhod denge**. Us aakhiri element ko store karke delete kar denge aur fir dono queues ko swap kar denge. → **Time: $O(N)$**

Approach 2: Poora System $O(1)$ Kaise Karein? (The Ultimate Design Twist)

Bhai, interviewer ko agar ek dum hairan karna hai, toh unhe ye **Amortized $O(1)$** approach batao. standard queue me `push` aur `pop` hamesha $O(1)$ hote hain, toh agar hum elements ko ghumaaye bina unka track rakhein, toh dono $O(1)$ ho sakte hain.

Hum single queue use karenge, par uske saath ek **topElement** variable maintain karenge.

Khel Kaise Chalta Hai?

- **push(x)**: Element ko seedha queue me daal do. Aur **topElement = x** set kar do. (Kyunki jo sabse aakhiri me aaya, wahi stack ka top hoga). → **Time: $O(1)$**
- **top()**: Kuch ghumane ki zaroorat nahi hai, seedha **topElement** variable return kar do. → **Time: $O(1)$**
- **pop()**: Sirf pop karte waqt hum queue ko $O(N)$ baar ghumayenge taaki agla top element mil sake.

"Bhai, par ye toh pichle jaisa hi hua?"

Nahi! Dhyan se dekho: Agar tumhare system me **1000 baar Push** ho raha hai aur sirf **1 baar Pop**, toh pichli approach me tumne 1000 baar loop chalaya ($1000 \times N$). Par is approach me Push ke waqt koi loop nahi chala ($O(1)$), loop sirf us 1 baar Pop ke waqt chalega! Isko bolte hain **Amortized $O(1)$ Complexity**—yani average me poora code $O(1)$ chal raha hai.

Code: Push $O(1)$ aur Pop $O(N)$ (Amortized Optimal)

```
#include <iostream>
#include <queue>
using namespace std;

class MyStackPushOptimal {
private:
    queue<int> q1; // Main Queue
    queue<int> q2; // Helper Queue
    int topElement; // Stack ke top ko  $O(1)$  me track karne ke liye

public:
    MyStackPushOptimal() {}

    // 1. PUSH:  $O(1)$  Time 🔥 (Ek dum Fast)
    void push(int x) {
        q1.push(x);
        topElement = x; // Har naya element hi stack ka top banega
    }

    // 2. POP:  $O(N)$  Time (Saara load yahan daal diya)
    int pop() {
        if (q1.empty()) return -1;

        // Jab tak q1 me sirf Ek element na bache, baaki sab q2 me daal do
        while (q1.size() > 1) {
            topElement = q1.front(); // Yeh naya topElement track hota rahega
            q2.push(q1.front());
            q1.pop();
        }

        // Jo aakhiri element bacha, wahi hamara pop hone wala element hai
        int poppedValue = q1.front();
```

```

    q1.pop(); // Use permanent delete kar diya

    // Queues ko swap kar diya taaki q1 fir se main queue ban jaye
    swap(q1, q2);

    return poppedValue;
}

// 3. TOP: O(1) Time 🔥 (Bina kisi loop ke direct variable se)
int top() {
    if (q1.empty()) return -1;
    return topElement;
}

// 4. EMPTY: O(1) Time
bool empty() {
    return q1.empty();
}

};

int main() {
    MyStackPushOptimal st;
    st.push(10); // O(1)
    st.push(20); // O(1)
    st.push(30); // O(1)

    cout << "Stack Top: " << st.top() << endl; // Output: 30 (O(1))
    cout << "Popped: " << st.pop() << endl;    // Output: 30 (O(N) chala yahan)
    cout << "New Stack Top: " << st.top() << endl; // Output: 20 (O(1))

    return 0;
}

```

Final Comparison Sheet (Interviewer ke Saamne Frame Karne ke Liye)

Approach	Push Time	Pop Time	Top Time	Space Complexity	Best Used When?
Brute Force (2 Queues)	$O(N)$	$O(1)$	$O(1)$	$O(N)$	Basic understanding ke liye
Optimal (1 Queue - Rotate)	$O(N)$	$O(1)$	$O(1)$	$O(1)$ Extra	Jab Pop operations bohot frequent hon
Push Optimal (Top Variable)	$O(1)$	$O(N)$	$O(1)$	$O(N)$	Jab Push operations bohot frequent hon (Amortized $O(1)$)

Bhai, isse zyada optimize is question ko pure computer science me nahi kiya ja sakta. Agar tum interviewer ko ye dono options de doge ki "Sir, aapko application me PUSH fast chahiye ya POP fast?"—toh wo tumhari design thinking dekh kar flat ho jayega!

Ab bolo, ye dono twists dimaag me fit baithe?

Bhai, "Implement Stack using Queue" ke dono approaches (Push-Optimal aur Pop-Optimal) par interviewer jab edge cases fekta hai, toh wo yeh dekhna chahta hai ki memory corrupt hogi ya nahi, aur variables (`topElement`) sahi se update ho rahe hain ya nahi.

Chalo, dono optimal approaches (`1-Queue Rotation` aur `Push-Optimal with top variable`) ke saare edge cases ka detailed dry-run dekhte hain taaki tumhara logic hamesha bulletproof rahe.

1. Underflow Case (Khaali Stack Me Pop Ya Top Karna)

Situation: Stack ekdum khaali hai, aur user ne direct `st.pop()` ya `st.top()` call kar diya.

1-Queue (Pop-Optimal) Me Kya Hoga?

```
if (q.empty()) return -1;
```

- **Result:** Code queue ke khaali hote hi pehli line se `-1` return kar dega. Koi invalid memory access nahi hoga. (PASS )

Push-Optimal (Top Variable) Me Kya Hoga?

Yahan par ek chota sa trap hai. Agar stack khaali hai aur hum direct `topElement` return kar dein, toh wo garbage value de sakta hai. Isliye code me ye check zaroori hai:

```
int top() {  
    if (q1.empty()) return -1; // Yeh check bacha leta hai  
    return topElement;  
}
```

- **Result:** Jaise hi `q1.empty()` true hoga, ye `-1` de dega aur safe exit karega. (PASS )
-

2. Single Element Case (Stack Me Sirf Ek Hi Element Hai)

Situation: `st.push(10)` kiya, aur phir turant `st.pop()` ya `st.top()` call kiya.

1-Queue (Pop-Optimal) Me:

- `push(10)` chalne par `currentSize = 0` (kyuki queue pehle khaali thi).
- `q.push(10)` hua. Loop `0` baar chalega (yani rotation nahi hoga). Queue me `[10]` bacha.
- `st.top()` karne par direct `q.front()` se `10` mil gaya. (PASS )

Push-Optimal Me:

- `push(10)` karne par `q1.push(10)` hua aur `topElement = 10` set ho gaya.
- `pop()` call hone par: `while(q1.size() > 1)` loop chalega hi nahi, kyuki size pehle se 1 hai!
- Loop ke baad direct `poppedValue = q1.front()` (yaani 10) niklega aur `q1.pop()` se queue khaali ho jayegi.
- **Result:** Bina kisi extra loop ke single element sahi se pop ho gaya. (PASS ✓)

3. Duplicate Elements Case (`push(10)`, `push(10)`, `push(20)`)

Situation: Stack me baar-baar same values push ho rahi hain.

1-Queue (Pop-Optimal) Me:

- Queue ka rotation elements ki value par nahi, balki unke **count** (`currentSize`) par chalta hai. Isliye duplicate elements aapas me bina kisi glitch ke sahi se rotate ho kar stack ka LIFO order maintain rakhenge. (PASS ✓)

Push-Optimal Me:

- Jab `st.push(10)` fir `st.push(10)` hoga, toh `topElement` update hoke 10 hi rahega.
- Jab `pop()` chalega, toh pehla 10 `q2` me transfer ho jayega aur `topElement` wapas 10 set ho jayega. Dusra 10 pop ho kar nikal jayega. Duplicates se variable tracking par koi asar nahi padta. (PASS ✓)

4. Continuous Pop Operations (Jab Tak Stack Khaali Na Ho Jaye)

Interviewer ka Chhupa Trap (Push-Optimal Ke Liye): "Bhai, agar mai lagatar pop karta rahu, toh kya tumhara `topElement` variable sahi se piche wale elements ko track kar payega?"

Maan lo stack me `[30, 20, 10]` hain (30 top par hai). `topElement` abhi 30 hai.

- **Pehla Pop Call:** * Loop chalega aur 30 ko chhod kar baaki sab `q2` me jayenge.
- Jab 10 transfer hoga, `topElement = 10` hoga.
- Jab 20 transfer hoga, `topElement = 20` ho jayega!
- 30 delete ho gaya. Swapping ke baad ab stack me `[20, 10]` bache aur `topElement` automatic update hokar 20 ho gaya!
- **Doosra Pop Call:**
- Loop chalega, 10 transfer hoga aur `topElement = 10` ho jayega. 20 delete ho gaya.
- **Result:** Loop ke andar chalne wali line `topElement = q1.front()` har ek step par piche wale elements ka naya track banti chalti hai. Isliye lagatar pops karne par bhi variable kabhi galat value nahi dikhayega. (PASS ✓)

Edge Case Matrix (Quick Revision)

Scenario	Trap/Doubt	1-Queue Rotation	Push-Optimal (Top Variable)
Khaali Stack (Underflow)	Garbage value ya Crash?	<code>if (q.empty())</code> bacha leta hai.	<code>if (q1.empty())</code> check safe exit deta hai.
Single Element	Infinite loops ka darr?	<code>size = 0</code> hone se loop nahi chalta, perfect.	<code>size > 1</code> ki condition fail hoti hai, loop bypass hota hai.
Duplicate Elements	Ordering mix-up?	Count-based loop hai, value se farq nahi padta.	Variable latest copy ko track karta hai, values sahi rehti hain.
Lagatar Pop Calls	<code>topElement</code> purani value toh nahi dikhayega?	N/A (Hamesha front se leta hai).	Loop ke andar har swipe par variable update hota rehta hai.

Bhai, ab dono me se koi bhi approach ho, uske saare edge cases aur unka internal working ekdum dhasu tarike se clear hai? Interviewer ab tumhe isme kahin nahi fasa sakta!

Implement Queue using Stack

Bhai, **Queue Using Stacks** ka **Brute Force Approach** ekdum straightforward hai. Iska main target hota hai: **"Push operation ko itna expensive ($O(N)$) bana do ki jab bhi koi naya element aaye, wo pehle se hi sahi FIFO (Queue) order me set ho jaye."** Isse fayda ye hota hai ki hamara `pop()` aur `peek()` ekdum makkhan ki tarah $O(1)$ me chalte hain.

Chalo iska poora logic aur code step-by-step samajhte hain.

Brute Force Ka Logic (Double Palti Trick)

Stack ka rule hota hai **LIFO** (Last In First Out) aur Queue ka hota hai **FIFO** (First In First Out). Agar hum ek stack ke saare elements ko nikaal kar dusre stack me daal dein, toh unka order ekdum ulta (reverse) ho jata hai. Aur agar hum unhe wapas pehle stack me daal dein, toh order fir se seedha ho jata hai. Yahi double palti ka khel hum yahan khelenge.

Hum do stacks use karenge:

1. `s1` (Hamara **Main Stack** - jisme hamesha final queue ka order maintain rahega).
2. `s2` (Hamara **Helper Stack** - jo naye element ko sahi jagah fit karne me madad karega).

Push Kaise Kaam Karta Hai? (Step-by-Step)

Maan lo `s1` me pehle se `10` aur `20` hain (Yahan `10` top par hai, kyunki queue me `10` pehle aaya tha). Ab naya element aaya `30`.

1. **Step 1:** `s1` ke saare elements ko ek-ek karke nikaalo (`pop`) aur helper stack `s2` me push karte jao.
 - `s2` dikhega aise: `[20, 10]` (Yahan `10` ekdum neeche chala gaya, aur `20` top par hai).
2. **Step 2:** Ab `s1` ekdum khaali ho chuka hai. Naye element `30` ko chupchap `s1` me push kar do.

- `s1` me ab sirf ek element hai: `[30]`. (Yeh ekdum base par baith gaya, kyunki queue me ye sabse aakhiri me aaya hai).
3. **Step 3:** Ab `s2` ke saare elements ko wapas nikaal kar `s1` me udel (pour) do.
- `s1` ab dikhega aise: `[10, 20, 30]` (Dhyan se dekho! `10` wapas top par aa gaya).

Magic! Kyunki `10` main stack `s1` ke top par hai, jab bhi koi `pop()` ya `peek()` maangega, hume direct $O(1)$ me mil jayega.

Brute Force C++ Code ($O(N)$ Push)

```
#include <iostream>
#include <stack>
using namespace std;

class MyQueueBrute {
private:
    stack<int> s1; // Main Stack (Hamesha Queue order hold karega)
    stack<int> s2; // Helper Stack

public:
    MyQueueBrute() {}

    // 1. PUSH Operation: O(N) Time (Saara load yahan hai)
    void push(int x) {
        // Step 1: s1 ke saare elements ko s2 me palti marwao
        while (!s1.empty()) {
            s2.push(s1.top());
            s1.pop();
        }

        // Step 2: Naye element ko khaali s1 me daalo (ye bottom banega)
        s1.push(x);

        // Step 3: s2 ke saare elements ko wapas s1 me daalo
        while (!s2.empty()) {
            s1.push(s2.top());
            s2.pop();
        }
    }

    // 2. POP Operation: O(1) Time 🔥
    int pop() {
        if (s1.empty()) return -1; // Underflow Check

        int frontElement = s1.top();
        s1.pop(); // Main stack ke top par hamesha sabse purana element hoga
        return frontElement;
    }
}
```

```
// 3. PEEK (Front Element): O(1) Time 🔥
int peek() {
    if (s1.empty()) return -1;
    return s1.top(); // Direct top return kiya
}

// 4. EMPTY Operation: O(1) Time
bool empty() {
    return s1.empty();
}

};

int main() {
    MyQueueBrute q;
    q.push(10);
    q.push(20);
    q.push(30);

    cout << "Queue Front: " << q.peek() << endl; // Output: 10 (FIFO Order)
    cout << "Popped: " << q.pop() << endl;        // Output: 10
    cout << "New Queue Front: " << q.peek() << endl; // Output: 20

    return 0;
}
```

Complexity Analysis

- **Time Complexity:**
- **push(x):** $O(N)$ ❌ (Kyunki har naye element ke liye hum poorie stack ko do baar palti marwate hain —pehle $s1$ se $s2$ me, fir $s2$ se $s1$ me. Total $2N$ operations hote hain, jise hum $O(N)$ likhte hain).
- **pop() / peek() / empty():** $O(1)$ 🔥 (Main stack ke top se direct kaam ho jata hai, koi loop nahi lagta).
- **Space Complexity:** $O(N)$
- **Kyun?** Kyuki humne N elements ko store karne ke liye do stacks ka use kiya hai. Bhalerao dono stacks milkar maximum N elements hi store karte hain, par unhe lagane ke liye auxiliary space utilize hoti hai.

Charo Edge Cases Par Iska Performance

1. Underflow Case (Khaali Queue me Pop/Ppeek):

- Code me check laga hai `if (s1.empty()) return -1;`. Queue khaali hote hi ye safely `-1` return kar dega aur code crash hone se bach jayega. **(PASS ✅)**

2. Ek Hi Element Hona:

- `push(10)` karne par `s1` pehle se khaali hai, toh pehla `while` loop nahi chalega. `10` seedha `s1` me push ho jayega. Dusra loop bhi nahi chalega. `s1` me perfectly `[10]` store ho jayega. **(PASS ✓)**

3. Duplicate Elements (`push(10)`, `push(10)`):

- Stacks elements ke duplicate hone se nahi, balki unke aane ke sequence par chalte hain. Dono `10` ek ke upar ek perfectly stack honge aur FIFO order maintain rahega. **(PASS ✓)**

4. Bade Inputs / Continuous Operations:

- Agar constraints chote hain (jaise $N \leq 100$), toh ye brute force perfectly pass ho jata hai. Lekin bade inputs par $O(N)$ push lagatar hone se code heavy pad sakta hai. **(PASS ✓ mathematically, par performance wise slow)**

Bhai, brute force ka poora khel samajh aa gaya? Ab iska better ($O(N)$ pop) aur optimal (Amortized $O(1)$) wala dimaag ekdum fresh hai ya unka bhi ek baar dry-run dekhna hai?

Bhai, ab aate hain is question ke **Sabse Khatarnak aur Absolute Optimal Approach** par, jise **Amortized $O(1)$ Time** approach bola jata hai.

Interview me interviewer brute force ke baad tumse yahi expect karega kyunki isme hum bina elements ko baar-baar aage-piche ghumaye, **Average $O(1)$ Time** me `push`, `pop`, aur `peek` teeno kaam kar lete hain. Hum elements ko tabhi transfer karte hain jab ekdum aakhiri mauka ho (**Lazy Evaluation**).

1. Optimal Logic: The Lazy System (Amortized $O(1)$)

Hum do stacks use karenge, par dono ka kaam ekdum divide kar denge:

1. **input Stack:** Naye elements ko receive karne ke liye (Hamesha $O(1)$ Push).
2. **output Stack:** Elements ko Queue (FIFO) order me nikalne ke liye.

Khel Kaise Chalta Hai? (The Golden Rule)

- **`push(x)`:** Kuch mat soch, naye element ko chupchap `input` stack me daal do. Baat khatam! → **Time: $O(1)$**
- **`pop()` / `peek()`:** Hum seedha `output` stack ke paas jayenge aur bolenge, "*Bhai tera top element de.*"
- **Case A (Agar `output` stack me pehle se maal hai):** Toh jo uske top par hai, wahi queue ka front hai. Use direct pop/peek kar lo. → **Time: $O(1)$**
- **Case B (Agar `output` stack ekdum KHAALI hai):** Tab hum `input` stack ke **saare elements** ko ek hi baar me udel (pour) denge `output` stack me. Palti marte hi saare elements ka order reverse ho jayega aur hume perfectly FIFO order mil jayega. Uske baad `output` ke top se element nikaal lo. → **Amortized $O(1)$**

2. Optimal C++ Code (Amortized $O(1)$)

```
#include <iostream>
#include <stack>
```



```
using namespace std;

class MyQueueOptimal {
private:
    stack<int> input;
    stack<int> output;

public:
    MyQueueOptimal() {}

    // 1. PUSH: Hamesha O(1) 🔥 (Ekdam Fast)
    void push(int x) {
        input.push(x);
    }

    // 2. POP: Amortized O(1) 🔥
    int pop() {
        if (empty()) return -1; // Underflow Check

        // Agar output stack khaali hai, tabhi input se saare elements transfer
        karo
        if (output.empty()) {
            while (!input.empty()) {
                output.push(input.top());
                input.pop();
            }
        }

        // Output ke top par hamesha sabse purana (FIFO) element milega
        int frontValue = output.top();
        output.pop();
        return frontValue;
    }

    // 3. PEEK (Front Element): Amortized O(1) 🔥
    int peek() {
        if (empty()) return -1;

        if (output.empty()) {
            while (!input.empty()) {
                output.push(input.top());
                input.pop();
            }
        }

        return output.top();
    }

    // 4. EMPTY: O(1)
    bool empty() {
        return input.empty() && output.empty();
    }
};
```

```
int main() {
    MyQueueOptimal q;
    q.push(10); // O(1)
    q.push(20); // O(1)
    q.push(30); // O(1)

    cout << "Queue Front: " << q.peak() << endl; // Output: 10 (FIFO Order)
    cout << "Popped: " << q.pop() << endl;        // Output: 10
    cout << "New Queue Front: " << q.peak() << endl; // Output: 20

    return 0;
}
```

3. Complexity Analysis (Interview Me Kaise Samjhayein?)

- **Push Complexity:** $O(1)$ 🔥
- **Pop/Peek Complexity:** Amortized $O(1)$ 🔥
- **Amortized ka kya matlab hai?** Interviewer ko bolo: "Sir, jab **output** stack khaali hota hai, tabhi hum $O(N)$ ka loop chalakar elements transfer karte hain. Par ek baar transfer hone ke baad, agle saare pop/peek operations direct $O(1)$ me hote hain jab tak **output** stack dobara khaali na ho jaye."
- Agar hum dhyan se dekhein, toh har ek element maximum **do baar push** hota hai (ek baar input me, ek baar output me) aur **do baar pop** hota hai. Toh total operations $4N$ hain N elements ke liye, yani average cost per element hamesha $O(1)$ aati hai!
- **Space Complexity:** $O(N)$ (Elements ko store karne ke liye do stacks use ho rahe hain).

4. Un Chaaro Edge Cases Par Iska Performance

Yeh lazy system bohot smart hai, dekho kaise har edge case ko lapet-ta hai:

1. Underflow Case (Khaali Queue me Pop/Peek):

- Code me check laga hai `if (empty()) return -1;` (jo check karta hai ki dono **input** aur **output** khaali hain ya nahi). Agar dono khaali hain, toh bina crash kiye **-1** de dega. **(PASS ✅)**

2. Lagatar Pops (**pop()**, **pop()**, **pop()**):

- Pehle pop par saara maal **output** me chala gaya. Agle pops ke waqt **output.empty()** false ho jayega, toh loop bypass ho jayega aur direct $O(1)$ me top element milta rahega. Ek dum makkhan! **(PASS ✅)**

3. Interleaved Operations (**push**, **pop**, **push**, **pop**):

- Maan lo **10** aur **20** push hue **input** me. Pop call hua, dono palti maar kar **output** me chale gaye (**[20, 10]**). **10** pop ho gaya.
- Ab agla **push(30)** aaya, toh **30** chupchap **input** me jaakar baith jayega, wo **output** ko bilkul chhedega bhi nahi.

- Agla `pop()` call direct `output` ke top se `20` ko nikaal lega. `30` tab tak wait karega jab tak `output` poora khaali nahi ho jata. FIFO order ekdum tight rehta hai! **(PASS ✓)**

4. Duplicate Elements:

- Palti marne par duplicates ka relative order hamesha reverse ho kar queue format me hi set hota hai, math me koi dikkat nahi aati. **(PASS ✓)**

Bhai, ab Queue using Stacks ka Brute ($O(N)$ Push) aur ye dhasu Optimal (Amortized $O(1)$) dono concepts poori tarah dimaag me chhap gaye?

Bhai, **Queue Using Stacks** ke is optimal **Amortized $O(1)$** code par interviewer tumhe phasane ke liye bohot hi tricky scenarios fek sakta hai. Wo check karega ki jab dono stacks aapas me coordinate kar rahe hain, toh kya kisi point par data mix-up ya order galat toh nahi ho raha.

Chalo, is lazy system ke **saare critical edge cases** ka poora breakdown dekhte hain taaki tumhara logic hamesha bulletproof rahe.

1. Interleaved Operations (Push aur Pop ka Mix)

Interviewer ka Fav Trap: "Bhai, agar mai 2 elements push karu, fir 1 pop karu, fir 1 naya element push karu, toh kya naya element purane element se pehle toh nahi pop ho jayega?"

Situation: `push(10)`, `push(20)`, `pop()`, `push(30)`, `pop()`

Dry Run:

- **`push(10)` aur `push(20)`:** Dono chupchap `input` stack me chale gaye.
- `input` = `[10, 20]` (20 top par hai), `output` = `[]`
- **Pehla `pop()`:** `output` khaali hai, toh `input` ka saara maal palti maar kar `output` me gaya.
- `input` = `[]`, `output` = `[20, 10]` (10 top par hai). `10` pop ho kar return ho gaya.
- **`push(30)`:** Yeh naya element chupchap `input` stack me chala gaya. Isne `output` ko bilkul touch nahi kiya!
- `input` = `[30]`, `output` = `[20]`
- **Dusra `pop()`:** Ab `output` khaali **nahi** hai (`[20]` baitha hai). Toh code bina `input` ko chhede direct `output` ke top se `20` ko nikaal dega!
- **Result:** Pehle `10` nikla, fir `20` nikla. `30` tab tak wait karega jab tak `output` poora khaali nahi ho jata. FIFO order ekdum perfect **PASS ✓** ho gaya.

2. Underflow Case (Khaali Queue Me Pop Ya Peek)

Situation: Dono `input` aur `output` stacks ekdum khaali hain, aur user ne direct `q.pop()` call kar diya.

Handling: Code me humne sabse upar ek master check lagaya hai:

```
if (empty()) return -1;
```

Yahan `empty()` function check karta hai: `return input.empty() && output.empty();`. Kyuki dono khaali hain, ye sach (`true`) return karega, aur hamara pop/peek bina kisi stack crash ya null pointer exception ke safely `-1` de dega. (PASS ✓)

3. Continuous Pop Operations (Jab Tak Queue Khaali Na Ho)

Situation: Maan lo humne teen elements push kiye `[10, 20, 30]` aur ab hum lagatar teen baar `pop()` call kar rahe hain.

Dry Run:

- **Pehla Pop:** `output` khaali tha, toh saare N elements transfer hue ($O(N)$ time laga). `output` ban gaya `[30, 20, 10]`. `10` nikal gaya.
 - **Dusra Pop:** `output` me `[30, 20]` bache hain. `output.empty()` false ho gaya, toh loop **nahi chalega**. Direct `20` mil gaya ($O(1)$ time).
 - **Teesra Pop:** `output` me `[30]` bacha hai. Loop fir nahi chalega, direct `30` mil gaya ($O(1)$ time).
 - **Result:** Pehle pop ko chhodkar baaki saare pops ek jhatke me $O(1)$ me chal gaye. Isiliye ise **Amortized** $O(1)$ bolte hain. (PASS ✓)
-

4. Single Element Case (Queue Me Sirf Ek Hi Element Hai)

Situation: `push(5)` kiya, aur phir turant `pop()` kiya.

Dry Run:

- `push(5)` karne par `input = [5]`.
 - `pop()` call hone par `output` khaali hai, toh `while(!input.empty())` loop chalega aur `5` transfer ho jayega `output` me.
 - `input` khaali ho gaya, `output = [5]`. Top element `5` pop ho kar safely bahar aa jayega. Loop sirf ek hi baar chala, koi overhead nahi hua. (PASS ✓)
-

5. Duplicate Elements Case (`push(10)`, `push(10)`, `push(20)`)

Situation: Same values baar-baar queue me aa rahi hain.

Reality: Stacks ko elements ki value se koi matlab nahi hota, wo sirf unke aane ke order (`top` positions) par kaam karte hain. Jab `input` se `output` me double palti lagti hai, toh duplicates ka relative order bhi perfectly queue ke FIFO नियम के मुताबिक ही set होता है. (PASS ✓)

Interview Cheat-Sheet: Edge Cases Verification

Scenario	Trap/Doubt	Code Kaise Handle Karta Hai?	Result
Khaali Queue (Underflow)	Crash ya Garbage Value?	<code>input.empty()</code> && <code>output.empty()</code> dono check hote hain, safe <code>-1</code> return.	PASS ✓
Mix Operations (push/pop)	Kya naya data order bigadega?	Naya data <code>input</code> me wait karta hai, jab tak <code>output</code> poora khaali na ho jaye.	PASS ✓
Lagatar Pops	Baar-baar transfer ka loop chalega?	<code>if (output.empty())</code> check loop ko block kar deta hai agar maal pehle se present ho.	PASS ✓
Single Element	Transfer loop phansega?	Loop sirf ek baar chal kar element shift kar deta hai, absolute safe execution.	PASS ✓

Bhai, ab is Amortized $O(1)$ Queue using Stacks ke saare edge cases, dry-runs aur core internals ekdum crystal clear hain? Interviewer ab tumhe is topic me bilkul nahi hila sakta!